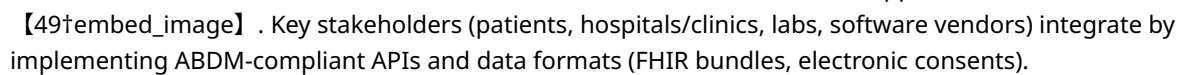


Ayushman Bharat Digital Mission (ABDM)

Sandbox Overview

The ABDM Sandbox provides a **developer environment** for India's digital health ecosystem. It introduces the **ABHA (Ayushman Bharat Health Account)** – a 14-digit Health ID for every Indian – and components like the **Health Information Exchange & Consent Manager (HIE-CM)**, Health Facility Registry (HFR), Health Professional Registry (HPR), and Unified Health Interface (UHI). Developers (e.g. hospitals or PHR app builders) can **register on HFR/HPR**, obtain sandbox credentials, and call ABDM APIs. The sandbox mimics production flows: users create an ABHA number, verify identity at a facility, and consent to data sharing. Figure below illustrates the high-level landscape of ABDM – linking health records to ABHA addresses via HIE-CM, with HFR/HPR for identities and PHR apps as HIUs

 . Key stakeholders (patients, hospitals/clinics, labs, software vendors) integrate by implementing ABDM-compliant APIs and data formats (FHIR bundles, electronic consents).

Milestone 1: ABHA Number Creation & Address Verification

Milestone 1 covers **issuing the digital health ID (ABHA)** and verifying a patient's identity/address at a health facility. Developers must integrate with ABDM's *ABHA Number APIs* to create an ABHA via Aadhaar OTP/biometric or other KYC (like driving licence) [\[14†L18-L26\]](#) . For example, when a patient gives consent, the hospital system calls ABDM (version 3) to **create an ABHA**; on success ABDM returns a 14-digit ID. The patient can then download their ABHA card or view it in the Sandbox ABHA App [\[14†L18-L26\]](#) . Next, on patient registration at the facility, the app/platform verifies that this patient really owns the ABHA. ABDM supports several verification flows:

- **Scan Health Facility QR:** The patient's ABHA QR code (or e-mail link) is scanned by the hospital's system, returning a *linking token* via HIE-CM, confirming identity.
- **By Demographics:** Without QR, the patient provides demographics (name, DOB, gender, address); the system calls ABDM APIs to match these and obtain the ABHA address with a link token [\[17†L19-L28\]](#) [\[17†L65-L74\]](#) .
- **By OTP:** The system can send a mobile OTP (via ABDM's */patients/otp/send* API) to the patient's registered mobile; upon entry of the OTP, ABDM validates it and returns the user's ABHA and KYC details [\[15†L17-L25\]](#) .
- **New vs Returning Patients:** The flow handles first-time vs repeat visits (e.g. avoid re-verifying an already-linked ABHA) [\[16†L18-L27\]](#) .

Upon successful ABHA verification, the hospital's software *stores* the ABHA address in the patient's record. If the patient linked their ABHA at registration, the hospital saves the **linking token** provided by ABDM for future record linking. These flows ensure **unique, authenticated patient IDs** across all providers. (See Milestone 1 test cases for detailed criteria.)

Milestone 2: Linking Health Records & Consent Preparation

Milestone 2 enables **linking medical records to ABHA addresses** and preparing for consent-based sharing. The sandbox docs specify:

- **Health Record Formats:** All clinical data must be in **FHIR** format (FHIR R4) as JSON. Supported Health Information (HI) types include Prescription, DiagnosticReport, OPConsultation, DischargeSummary, Immunization, HealthDocument, and WellnessRecord. Each must use the specified SNOMED-CT codes (e.g. "721981007" for DiagnosticReport) [38†L25-L34] . HIP systems should construct **FHIR Bundles** ("document" bundles) with a Composition resource and entries (e.g. a DiagnosticReport plus Observations) as shown in the examples [35†L9-L18] . Developers can reuse FHIR libraries and validators (see references on NRCeS site).
- **Care Contexts:** A **Care Context** is a logical container for a patient's health records at a facility (e.g. "IPD hospitalization Sep2023"). It has a display name and a reference ID. The system must organize records under care contexts so ABDM can group and share them correctly [22†L10-L19] .
- **Linking Records – HIP Initiated:** If the patient *already shared ABHA* during registration, the hospital (HIP) uses the saved linking token to attach new records to that ABHA. Whenever a health record is created, the hospital calls HIE-CM's "add-contexts" link API (V0.5 or V3) with the linking token. HIE-CM then confirms the linkage, and notifies any PHR apps subscribed for updates [25†L21-L28] . (Test cases: verify health record creation and linking via OTP or demographic.)
- **Linking Records – Notification to Mobile:** If the patient didn't share ABHA earlier but only gave mobile demographics, the HIP calls ABDM's `/patients/sms/notify` API when a new record is ready. ABDM sends the patient an SMS with a **deep-link** to a consent manager app. The user then creates (or retrieves) their ABHA address and is guided to discover and link the new record via the PHR app [26†L15-L21] . This ensures that even unregistered patients can later link their records.
- **Discovery & Link (User-initiated):** At any time, the patient's PHR app can *discover* records from hospitals they've visited. The user selects a facility (HIP) and asks HIE-CM to discover records for their ABHA or mobile number. HIE-CM forwards this to the HIP; the HIP implements a matching algorithm (match ABHA→mobile→MRN with demographic checks) [27†L51-L60] . It returns any care contexts found. The user then picks contexts to link; the HIP sends an OTP to confirm, and upon confirmation HIE-CM links the contexts to the user's ABHA [27†L100-L108] . This flow is critical for users who join ABDM after receiving care.
- **Consents Understanding:** ABDM mandates *user consent* before sharing data. An HIU (e.g. doctor or insurer) *requests* consent by calling HIE-CM with a signed consent request. The request includes purpose of use (from a set of codes like CAREMGT), date range, HI types needed, and data retention period [31†L12-L20] . The patient reviews the request in a consent app and grants or revokes it. ABDM's consent artefact (based on India's e-consent framework) is then signed by the patient. Upon grant, the HIE-CM notifies all relevant HIPs, which must store the consent artefact (or delete data on revoke) [29†L10-L18] [31†L39-L47] .

- **Storing Consent Artefacts:** Every HIP receiving data tied to a user's ABHA must save a copy of consents granted. When a consent is granted, the HIE-CM sends the HIP a notification (`POST /consents/hip/notify`); the HIP stores the consent (as per Test Cases). On revocation or expiration, the HIP must delete any stored data beyond the consent terms [29†L23-L27] .
- **Data Request (HIU side):** After consent, the HIU initiates a **health information request** via HIE-CM. This has 3 stages: (1) **Request:** HIU calls HIE-CM (`POST /health-information/hip/request`) with the consent ID, data-push URL, date-range and its public keys. HIE-CM returns a transaction ID. (2) **HIP validation & encrypt:** The HIP's repository checks the consent (valid, in date, types match), selects matching records, **encrypts** them with the HIU's public key, and *pushes* the data to the HIU's callback URL [32†L45-L53] [44†L43-L51] . (3) **Notifications:** The HIP then calls `POST /health-information/hip/notify` to inform HIE-CM it sent data. The HIU likewise notifies HIE-CM of successful receipt [32†L49-L53] [44†L55-L60] . This ensures secure, auditable transfer. (Data is encrypted with ECDH; see below.)
- **Data Packaging:** Health records are packaged as FHIR Bundles. The "Main Envelope" bundle's first entry is a Composition (document-level metadata), followed by each medical resource (e.g. DiagnosticReport, Observations) [21†L13-L21] . For example, a DiagnosticReport bundle includes the Composition then the DiagnosticReport and related Observations entries [35†L11-L20] . Each bundle must use the correct SNOMED-CT code for its type (e.g. code "721981007" for DiagnosticReport) [38†L29-L38] . Unstructured data (like PDFs) can be attached via `presentedForm` in FHIR. Developers should use FHIR client libraries, follow NRCeS guidance and validate bundles.
- **Encryption:** All transferred health data is **end-to-end encrypted**. ABDM uses Curve25519 Elliptic-curve Diffie-Hellman (ECDH) plus AES-GCM. The HIU generates a short-term key pair and nonce and includes these in the request to HIE-CM. The HIP then generates its own key pair and nonce, computes a shared session key via ECDH, and AES-encrypts the data [36†L33-L42] . The HIP sends the encrypted data along with its public key and nonce. The HIU uses its private key and the HIP's public key/nonce to derive the same session key and decrypt [36†L59-L68] . This ensures forward secrecy: compromising keys later cannot reveal past data. ABDM provides a **Fideliu CLI tool** to handle this encryption, generating key material and performing encrypt/decrypt without coding crypto from scratch [40†L9-L18] [40†L38-L47] . Using Fideliu is recommended for HIPs/HIUs; it simplifies handling of keys and AES-GCM encryption.
- **Testing Repositories:** The sandbox includes steps to validate integration. A test user installs the Consent Manager (ABHA Sandbox App), registers with a test patient's mobile, discovers facility records (using "Discover" in the app), and links care contexts via OTP [39†L42-L50] [39†L52-L60] . The developer (as an HIU) then logs into a reference HIU app, creates a consent request for that patient's ID, and the patient grants it. Finally the HIU fetches the data. This end-to-end scenario tests that your system correctly implements ABHA linking, consent handling, and data transfer [39†L69-L78] [39†L105-L113] .

Milestone 3: Consent Fulfillment & Record Retrieval (PHR/HIU)

Milestone 3 focuses on the PHR app / HIU workflows after a patient's data is linked: obtaining and displaying health records with consent.

- **Requesting Consent (HIU flow):** A PHR app or other HIU requests user consent by calling HIE-CM (`POST /consent-requests/init`) with the ABHA address, purpose (e.g. "CAREMG"), HI types needed and date range [41†L19-L27] . The gateway acknowledges with a request ID. HIE-CM notifies the patient's ABHA app of the pending request. The patient views it and grants or denies. HIE-CM then sends a notification back to the HIU (via `/consent-requests/on-init` callback) with the user's decision. If granted, it returns the ID(s) of generated consent artefact(s); if denied, it simply informs the HIU [41†L25-L32] . (E.g. a doctor's app must poll or listen for `on-init` and `on-status` callbacks to get the final consent status.)
- **Storing Consent Artefacts:** On receiving a *grant* notification, the HIU must fetch the actual consent artefact(s) by calling `/consents/fetch` with the consent ID [43†L51-L58] . These contain the signed details. The HIU stores these consent artefacts securely, as they are needed for future data requests. On *revocation or expiry*, HIE-CM calls the HIU's `/consents/hiu/notify` webhook [43†L13-L22] . The HIU must then delete any linked data beyond consent (per compliance) and acknowledge receipt.
- **Getting Health Records (HIU retrieval):** With a valid consent artefact, the HIU initiates data retrieval as in M2. The HIU's repository calls HIE-CM's `/health-information/hip/request` with the consent-artefact ID, time range, and its keys [44†L15-L24] . The rest of the flow mirrors Milestone 2: HIP validates and encrypts data, pushes it to the HIU's URL, and notifies HIE-CM [44†L43-L51] . Upon completion, the HIU receives the encrypted FHIR bundle and decrypts it.
- **Displaying Records (PHR App):** A PHR (Personal Health Record) app plays dual roles: it is a **HIU** when fetching data. Whenever a new care context is linked for a user, the app receives a notification. The PHR app should then automatically create a consent request for that context and send it to HIE-CM (the app's backend acts as HIU) [46†L15-L24] . If the user has enabled auto-approval, the consent is immediately granted. The app then requests the health record and stores it. The PHR UI should present the records chronologically (e.g. "My Records" tab) and allow the patient to view/manage consents [46†L15-L24] [46†L108-L117] . The docs outline features for a PHR: user login, profile management, notifications (for new record linking), subscription to facilities, handling deep-links, and maintaining an offline cache of records. Essentially, the PHR app must integrate all ABDM endpoints (ABHA APIs, HIE-CM APIs, etc.) to be "ABDM compliant."

Use Cases and Integration Components

Developers can use the sandbox to build **various applications**:

- **Hospital EHR/EMR systems (HIP):** Integrate ABHA creation into patient registration, call ABDM APIs for address verification and record linking, and implement the data-sharing APIs (link-care-context, consents, data-transfer) [25†L21-L28] [43†L15-L24] .

- **PHR apps (HIU/client apps):** Allow patients to create/manage their ABHA, register on HFR, discover/link their records, manage consents, and retrieve data from HIPs [46†L15-L24] [46†L108-L117] .
- **Health Information User systems (e.g. clinics, labs):** Request user consent via HIE-CM, fetch records when granted, and incorporate them into workflows (e.g. insurance, referrals).
- **Digital health services (UHI Integrators):** The Unified Health Interface will rely on ABDM IDs and registries; sandbox provides flows for ID linkage (see UHI docs, outside scope here).

Integration points include: the ABDM **Gateway** (network facade for APIs), the **HIE-CM** (core consent manager), the **Fideliu**s encryption tool, and local FHIR repositories. Developers must register for sandbox client IDs, set up callback URLs for their systems, and use provided Postman collections or reference apps to test.

Problems Solved by ABDM

ABDM addresses key issues in Indian healthcare interoperability:

- **Fragmented Records:** Patients' data is scattered across disparate systems. ABDM's linking and discovery means one PHR app can pull all records into a unified view.
- **No unified ID:** By issuing an ABHA number, it ensures every individual has a unique, portable health identity. This avoids duplicate records and enables interoperability.
- **Patient consent & privacy:** Data sharing is explicitly user-approved (aligned with PDPA). Electronic consents give patients control, and HIPs cannot share data without consent. Revocation is enforced, enhancing privacy.
- **Security of sensitive data:** The mandated end-to-end encryption (ECDH/AES-GCM) ensures that health data is protected in transit. Sandboxing and FHIR standards promote safe, standardized exchanges.
- **Discoverability of providers:** HFR and HPR provide trusted registries of facilities and professionals, so that apps can verify providers and patients can find nearby services.

Implementation Plan & Flows to Build

To implement an ABDM-compliant system, a team should plan in phases aligned with Milestones:

1. **Sandbox Setup & Learning:** Sign up at the ABDM sandbox portal. Obtain client IDs for HIP and/or HIU roles. Set up key pairs for encryption (or plan to use Fideliu's CLI). Review API specifications and examples from the sandbox docs.
2. **Implement Milestone 1 Flows (ABHA):**
3. Build features to **create an ABHA** (e.g. call ABDM's /v3/abha-number APIs with Aadhaar OTP/biometric flows) [14†L18-L26] . Test by simulating patient signup.
4. Integrate **ABHA address verification** at patient registration: e.g. scan QR (/auth/patients/upi/verify), or demographics-OTP flows [17†L19-L28] [15†L17-L25] . Ensure linking tokens are captured.
5. Test with the Sandbox ABHA app and provided Postman collections.

6. Implement Milestone 2 Flows (Linking & Data Prep):

7. **Health Records:** Adopt FHIR R4. Generate sample bundles (DiagnosticReport, etc.) following the provided examples [35†L9-L18] . Ensure to include Composition and proper coding.
8. **Linking:** After patient visits, use the linking token to call HIE-CM's link API for new records (HIP-initiated) [25†L21-L28] . Also implement SMS notification flow by calling `/patients/sms/notify` when ABHA absent [26†L15-L21] .
9. **Discovery:** Provide an interface for the PHR or HIP to initiate discovery; implement the `/care-contexts/discover` endpoint on HIP to return care contexts as per matching logic [27†L51-L60] .
10. **Consent Storage:** On receiving consent grant notifications from HIE-CM, store the consent artefact JSON as per [Understanding Consents] and [Store Consent] docs [31†L39-L47] [29†L10-L18] .
11. **Data Packaging & Encryption:** Before sharing, bundle the patient's records (within the requested date range and HI types) into a FHIR document bundle. Encrypt using ECDH: either implement curve25519 yourself or use **Fidelius CLI** (recommended) to encrypt the bundle with the HIU's public key [36†L33-L42] [40†L9-L18] . Test the encryption/decryption steps thoroughly.
12. **Data Sharing:** Implement the `/health-information/hip/request` and `/notify` callbacks. On health info request, validate consent, prepare and encrypt data, post to HIU's callback URL, then call `/health-information/hip/notify` [28†L15-L24] [32†L45-L53] .

13. Implement Milestone 3 Flows (HIU Data Retrieval):

14. **Consent Requests:** In the PHR (HIU) app or client, implement the ability to request consent: call `/consent-requests/init` with ABHA and consent details [41†L19-L27] . Handle the callback `/consent-requests/on-init` to receive consent IDs.
15. **Storing & Fetching Consent:** On grant notification, call `/consents/fetch` to retrieve the actual consent artefact. Store the `consentId` . On revoke/expiry, remove data as required [43†L13-L22] .
16. **Getting Records:** Call `/health-information/cm/request` (or `/hip/request`) with the `consentId` and your public key parameters [44†L15-L24] . Implement a webserver to receive the `/datapush-url` POST with encrypted data, then decrypt it (using Fidelius if used).
17. **Display UI:** Build the PHR app screens: "My Records" to list retrieved records, "Linked Facilities" showing where data came from, and "Consents" to show active/expired requests. On new notifications (linking or consent), prompt user or auto-handle as per settings [46†L15-L24] .
18. **Testing & Certification:**
19. Execute the provided **sandbox test cases** for Milestones 1/2/3 (see Test Cases sections) to ensure compliance. For example, verify that data cannot be fetched without consent or after revocation.
20. Exit the sandbox: after passing functionality and security tests, follow the **Sandbox Exit** process [11†L6-L15] . This involves STQC/CERT-IN security audit, documentation of APIs used, and coordination with NHA for production credentials.

Throughout, adhere to **security and compliance**: encrypt all sensitive data, validate all inputs, and follow PDPA guidelines for consent. Use the provided Postman collections and open-source Fidelius tool to accelerate development.

In summary, the ABDM sandbox documentation covers the full journey from patient ID creation through record exchange. Developers can build **hospital systems** that issue and verify ABHAs, and **PHR apps/HIUs** that manage consents and fetch data. The flows ensure patients own their health data (via ABHA and consents), and providers can securely interoperate using standard APIs and encryption [\[14†L18-L26\]](#) [\[36†L33-L42\]](#) .

Sources: ABDM Sandbox documentation (Milestones 1–3, PHR App Guide) [\[14†L18-L26\]](#) [\[26†L15-L21\]](#) [\[27†L51-L60\]](#) [\[32†L45-L53\]](#) [\[36†L33-L42\]](#) [\[40†L9-L18\]](#) [\[41†L19-L27\]](#) [\[43†L13-L22\]](#) [\[46†L15-L24\]](#) [\[47†L6-L14\]](#) and Sandbox Exit guidelines [\[11†L6-L15\]](#) .
